

Computer Science & IT



Compiler Design



Code Optimisation

Lecture No. 01



By- Ravindra Sir

Topics to be Covered



Topic

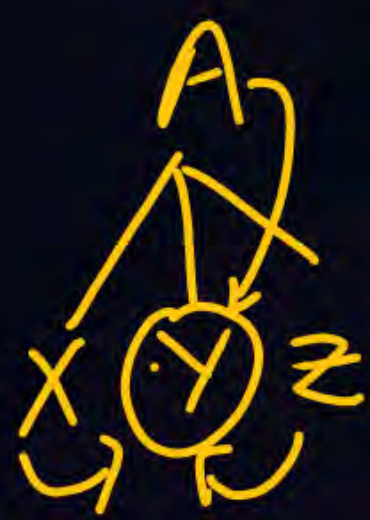
Topic

Topic

SDT

Inherited attribute

The value of an inherited attribute is computed from the values of attributes at the siblings and parent of that node



SOT for evaluating of an expression:-

$$E \rightarrow E_1 + T \quad \{ E.val = E_1.val + T.val \}$$

$$/ T \quad \{ E.val = T.val \}$$

$$T \rightarrow T_1 * F \quad \{ T.val = T_1.val * F.val \}$$

$$/ F \quad \{ T.val = F.val \}$$

$$F \rightarrow num \quad \{ F.val = num.lval \}$$

$$\underline{2 + 3 * 4}$$

SOT to convent infix to postfix:-

$E \rightarrow E + T \{ \text{Print '+'} \}$

$/ T \{ \}$

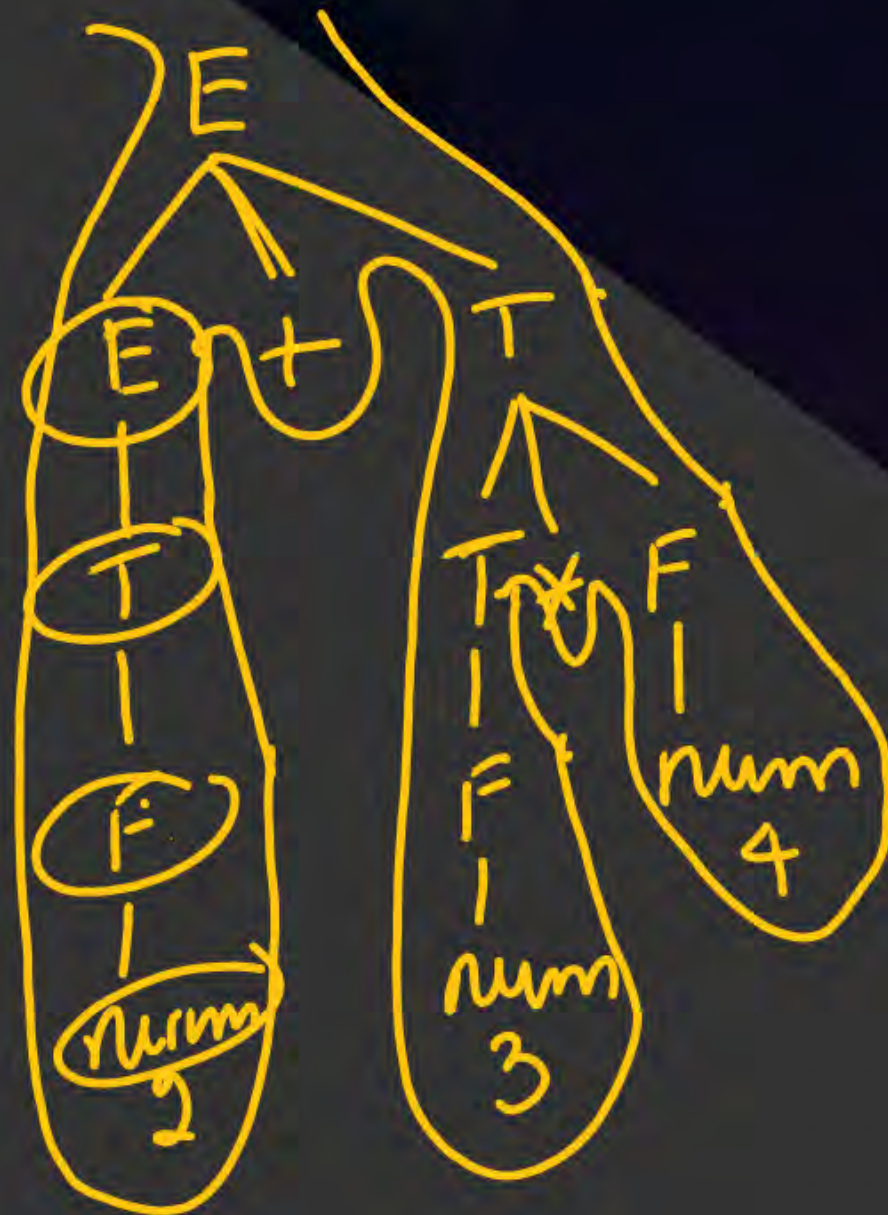
$T \rightarrow T * F \{ \text{Print '*' } \}$

$/ F \{ \}$

$F \rightarrow \text{num} \{ \text{Print}(\text{num.lval}) \}$

$2 + 3 * 4$

$234*+$



$S \rightarrow xxw$ (print '1')
 $\quad \quad \quad / y$ (print '2')
 $w \rightarrow sz$ { print '3' }

$xxxyzz$

23131



Gate 1

$$E \rightarrow E_1 * T \quad \{ E.val = E_1.val * T.val \}$$

$$/ T \quad \{ E.val = T.val \}$$

$$T \rightarrow F - T_1 \quad \{ T.val = F.val - T_1.val \}$$

$$/ F \quad \{ T.val = F.val \}$$

$$F \rightarrow 2 \quad \{ F.val = 2 \}$$

$$/ 4 \quad \{ F.val = 4 \}$$

$$\left. \begin{array}{l} \rightarrow * \\ \leftarrow - \end{array} \right\} \left((4 - (2 - 4)) * 2 \right)$$

(12)

$$E \rightarrow E_1 \# T \quad \{ E.val = E_1.val * T.val \}$$

$$/ T \quad \{ E.val = T.val \}$$

$$T \rightarrow T_1 \& F \quad \{ T.val = T_1.val + F.val \}$$

$$/ F \quad \{ T.val = F.val \}$$

$$F \rightarrow num \quad \{ F.val = num.lval \}$$

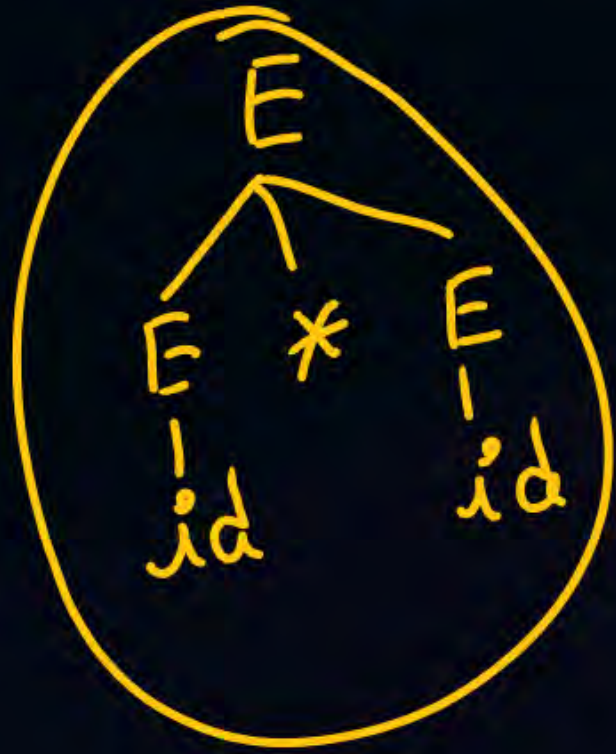
$$2 \# 3 \& 5 \# 6 \& 4$$

$$\begin{array}{c} * \\ \times \end{array} \quad \begin{array}{c} + \\ + \end{array} \quad \begin{array}{c} * \\ * \end{array} \quad \begin{array}{c} + \\ + \end{array}$$

$$2 * (3 + 5) * (6 + 4)$$

$$(2 * 8) * 10 = 160 \quad \checkmark$$

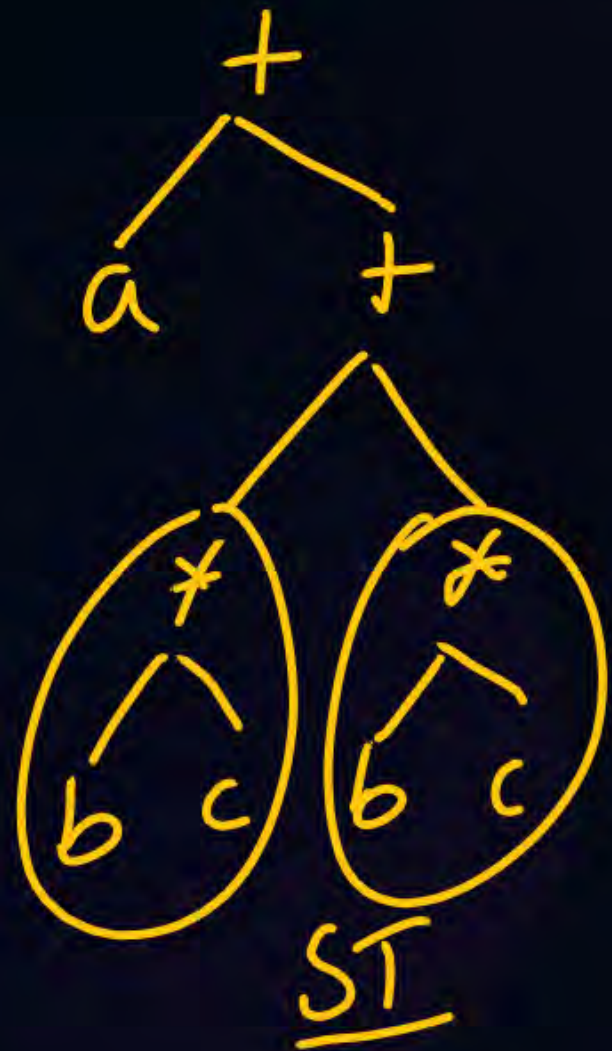
STD to build Syntax tree:-



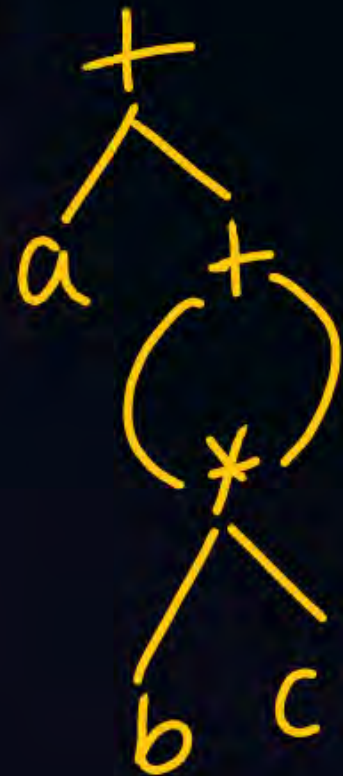
Directed acyclic graph:-

DAG is used to identify Common Sub expressions

It helps in eliminating redundant code



Das



Types of SDT

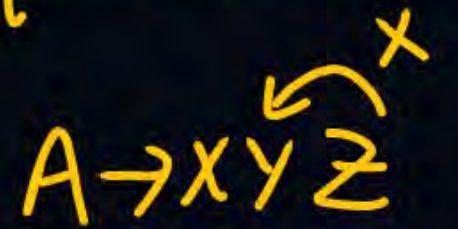
(S) attributed SDT

- 1) It uses only Synthesized attributes



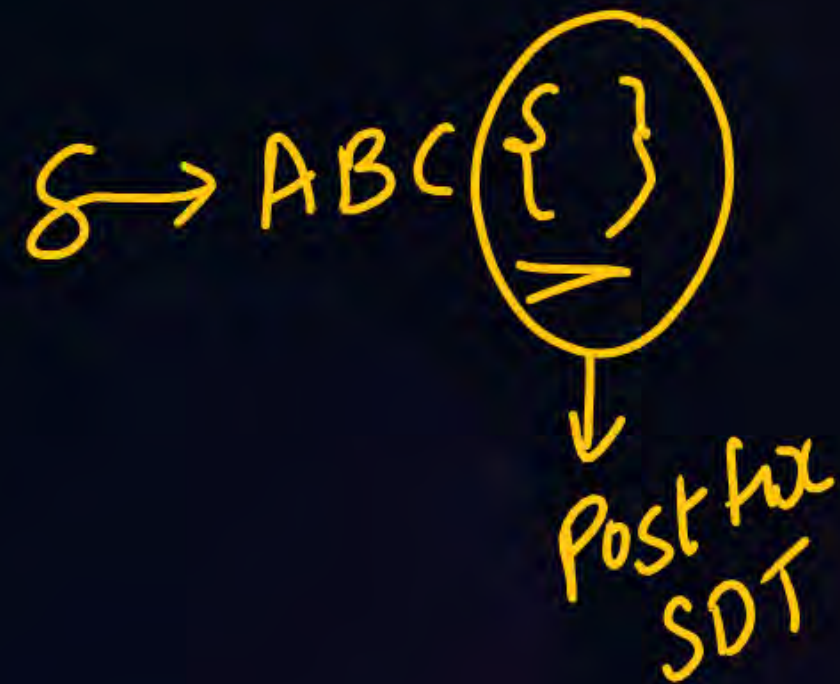
(L) attributed SDT

- 1) It allows both Synthesized and Inherited attributes. But if an Inherited attribute is present, then it is restricted ~~to~~ that it can inherit from the parent or left-siblings



S attn

2) Semantic actions can be placed at the end of the RHS of a production



L attn

2) Semantic actions can be placed anywhere on the RHS

$S \rightarrow A \{ \} B C$

S attr

3) Attributes are generally evaluated during BUP

L attr

3. Attributes are evaluated by traversing the parse tree depth first and left to right

Top down evaluation of a S- attributed grammar:-

$S \rightarrow (S_1) \{ S.count = S_1.count + 1 \} \#_1$
 $/ \in \{ S.count = 0 \} \#_2$

$S \rightarrow (S_1) \#_1$ $/ \#_2$	$($	$)$	$\$$
S	$S \rightarrow (S_1) \#_1$	$S \rightarrow \#_2$	$S \rightarrow \#_2$

$((())) \$$
 $\uparrow \uparrow \uparrow \uparrow \uparrow \uparrow$

③ $S.count = 0 \neq 3$

③ ✓

$\$$	$\$$	$\#_1$	$)$	S_1	$($	$\#_1$	$)$	S_1	$($	$\#_1$	$)$	S_1	$($	$\#_2$
------	------	--------	-----	-------	-----	--------	-----	-------	-----	--------	-----	-------	-----	--------

Consider following S-att definition for converting infix to postfix

$E \rightarrow E + T \{ \text{look } + \}$
 $/ T \{ \}$
 $T \rightarrow T * F \{ \text{look } * \}$
 $/ F \{ \}$
 $F \rightarrow id \{ \text{look } id \}$

5 min

$E \rightarrow E + T \#_1$
 $/ T$
 $T \rightarrow T * F \#_2$
 $/ F$
 $F \rightarrow id \#_3$

eliminate
left recursion

$a + b * c \rightarrow \text{postfix}$ Parse tree

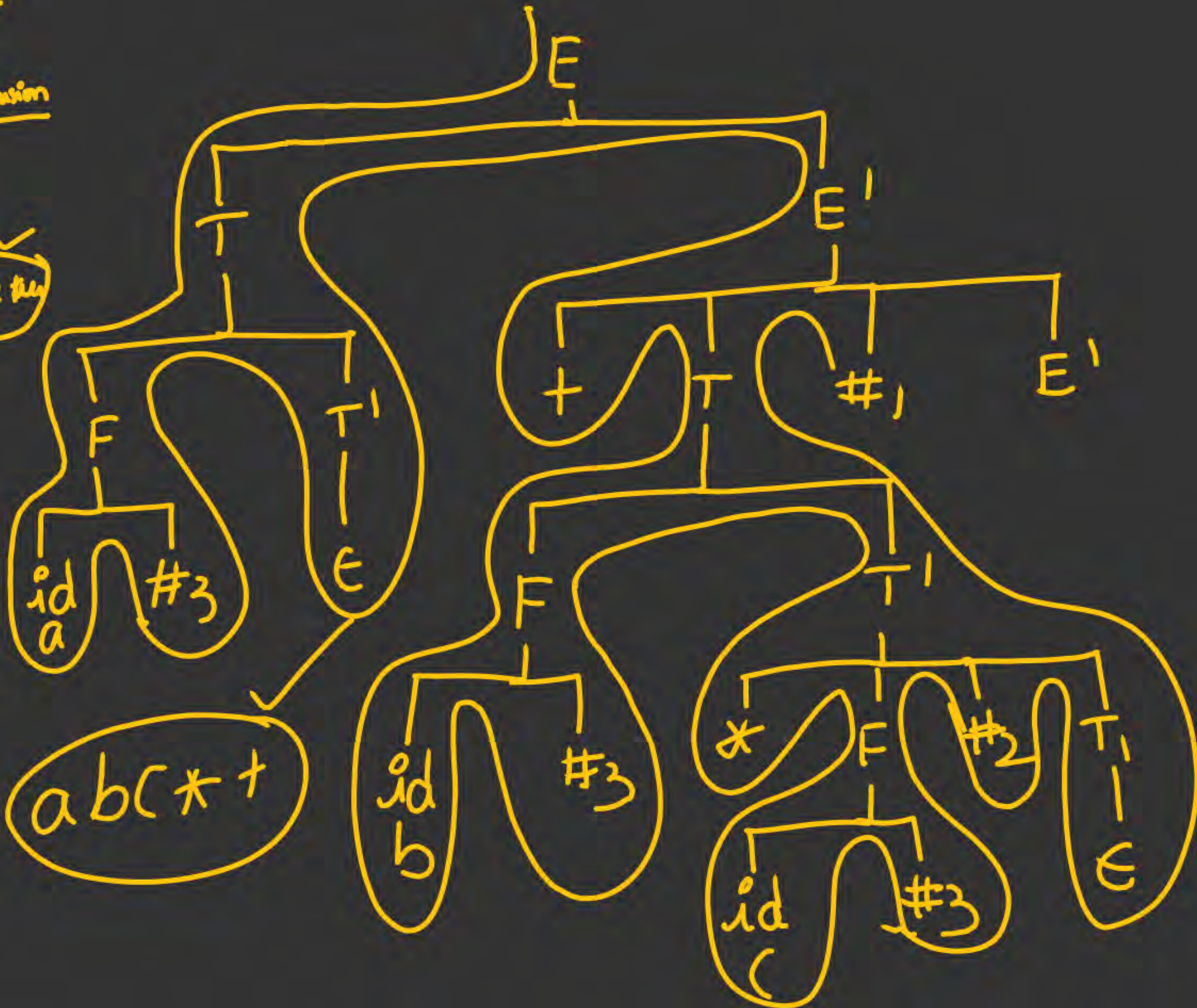
$E \rightarrow TE'$

$E' \rightarrow +T\#, E' / \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *F\#_2 T' / \epsilon$

$F \rightarrow id \#_3$



SOT for storing type information L all



✓
int a, b, c

5min

a	int
b	int
c	int

$D \rightarrow T \underline{L} \quad \{ L.Type = T.type \}$

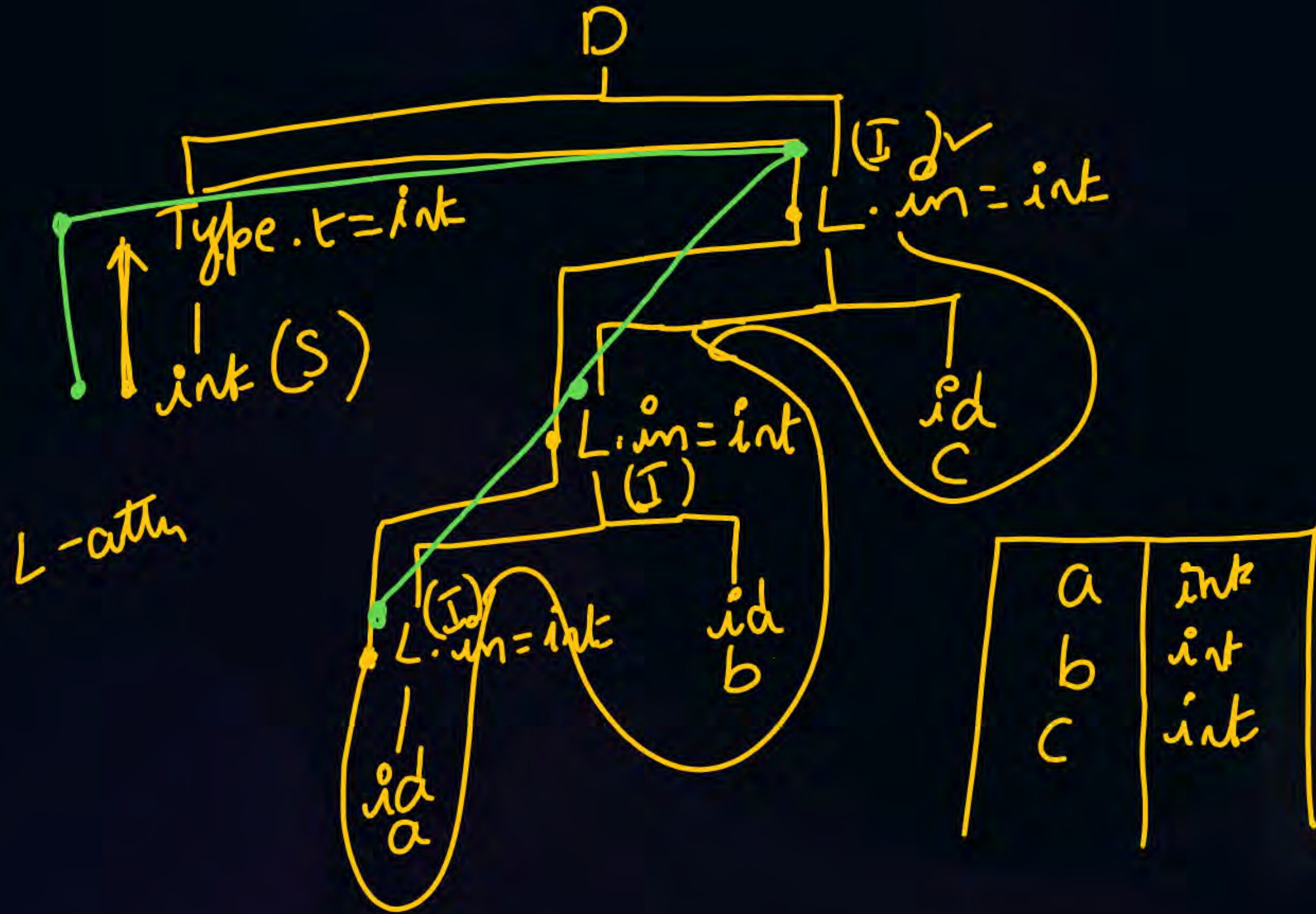
$T \rightarrow int \quad \{ T.Type = int \}$

$T \rightarrow char \quad \{ T.type = char \}$

$\underline{L} \rightarrow \underline{L_1}, id \quad \{ L_1.type = L.type \}$

$\{ \underline{add\ type(L_1.in, id)} \}$

$L \rightarrow id \quad \{ \underline{add\ type(L.in, id.name)} \}$



S-attr

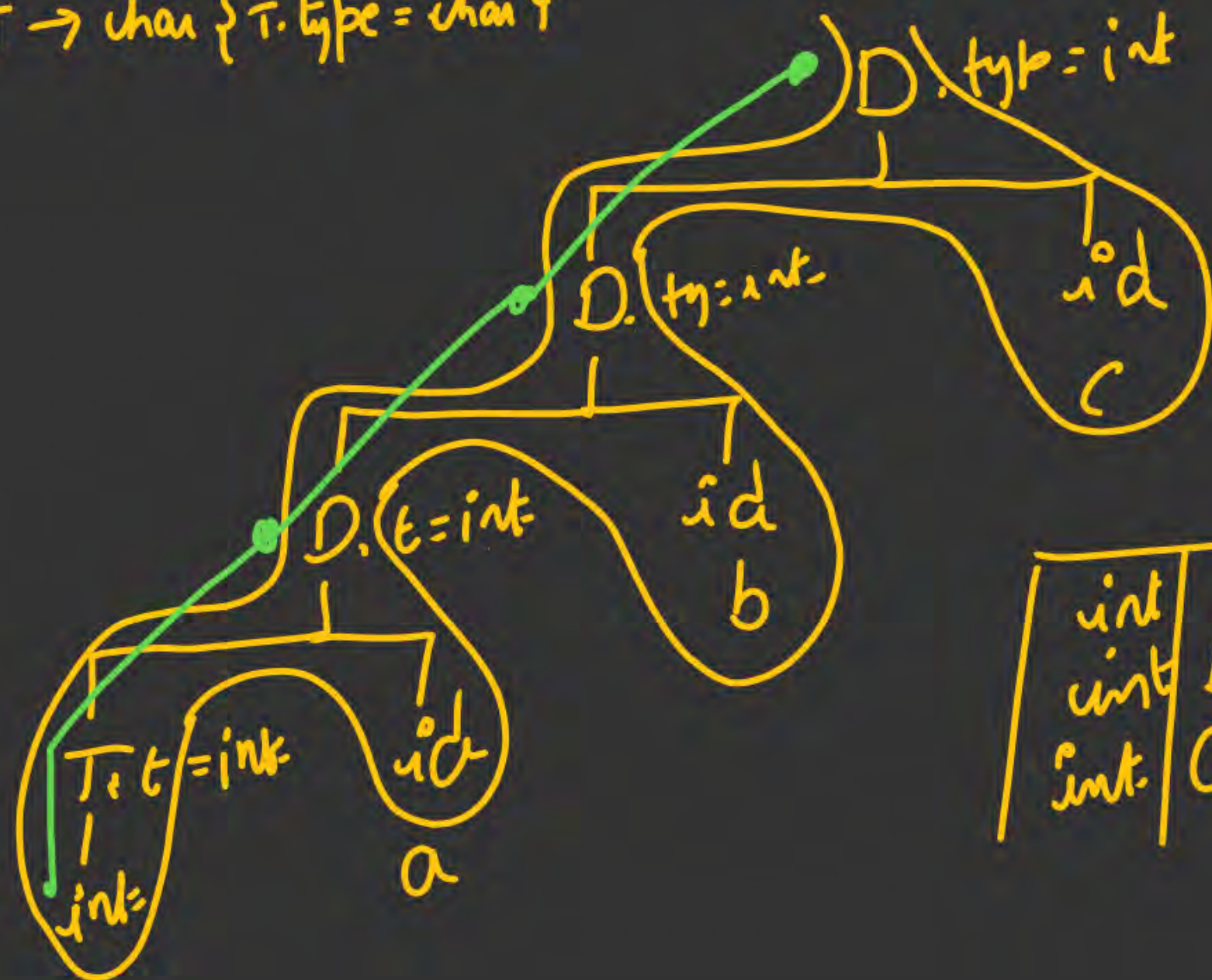
$P \rightarrow D, id \{ D.type = D.type; add_type(D.type, id.name) \}$

$D \rightarrow T, \underline{id} \{ D.type = T.type, add_type(T.type, id.name) \}$

$T \rightarrow int \{ T.type = int \}$

$T \rightarrow char \{ T.type = char \}$

int a, b, c



int	a
int	b
int	c

SOT for type checking:-



$E \rightarrow E_1 + E_2 \{ \text{if } (E_1.\text{type} == E_2.\text{type}) \ \&\& \ (E_1.\text{type} = \text{int}) \}$
/ $E_1 == E_2 \{ \text{if } (E_1.\text{type} == E_2.\text{type}) \ \&\& \ (E_1.\text{type} = \text{int} / \text{boolean}) \}$
/ $(E_1) \{ E.\text{type} = E_1.\text{type} \}$
/ $\text{num} \{ E.\text{type} = \text{int} \}$
/ $\text{True} \{ E.\text{type} = \text{bool} \}$
/ $\text{false} \{ E.\text{type} = \text{boolean} \}$

Binary number $\rightarrow$ SDT to count no of 1's
 SDT to count no of 0's
 " " " bits
 Binary to Decimal

	Count 1s	Count 0s	Count bits
$N \rightarrow L$	$N.C = L.C$	$N.C = L.C$	$N.C = L.C$
$L \rightarrow L_1 B$	$L.C = L_1.C + B.C$	$L.C = L_1.C + B.C$	$L.C = B.C + L_1.C$
$/B$	$L.C = B.C$	$L.C = B.C$	$L.C = B.C$
$B \rightarrow 0$	$B.C = 0$	$B.C = 1$	$B.C = 1$
$/1$	$B.C = 1$	$B.C = 0$	$B.C = 1$

Binary to decimal

$$0.1 \overset{2^{-1}}{\quad} \\ \textcircled{0.5}$$

$$0.11 \overset{2^{-1}}{\quad} \overset{2^{-2}}{\quad} \\ 0.5 \\ 0.25 \\ \hline 0.75$$

$$0.\underline{1101} \\ \frac{13}{24}$$

$$0.\textcircled{111} \Rightarrow \textcircled{\frac{7}{2^3}}$$

$$0.\textcircled{101} \\ \frac{5}{2^3}$$

$$N \rightarrow L_1 \cdot L_2 \left\{ N.d = L_1.dval + \frac{L_2.dval}{2^{L_2.count}} \right\}$$

$$L \rightarrow L_1 B \left\{ L.d = 2 * L_1.d + B.d, L.c = L_1.c + B.c \right\}$$

$$/ B \left\{ L.c = B.c, L.v = B.v \right\}$$

$$B \rightarrow 0 \left\{ B.d = 0, B.c = 1 \right\}$$

$$/ 1 \left\{ B.d = 1, B.c = 1 \right\}$$

$$\begin{array}{c} L \quad B \\ \downarrow \quad \downarrow \\ \underline{dx2 + B} \end{array}$$

SOT to generate 3 add code

$$x = a + b * c$$

$S \rightarrow id = E \{ \text{gen}(id.name = E.place), \}$

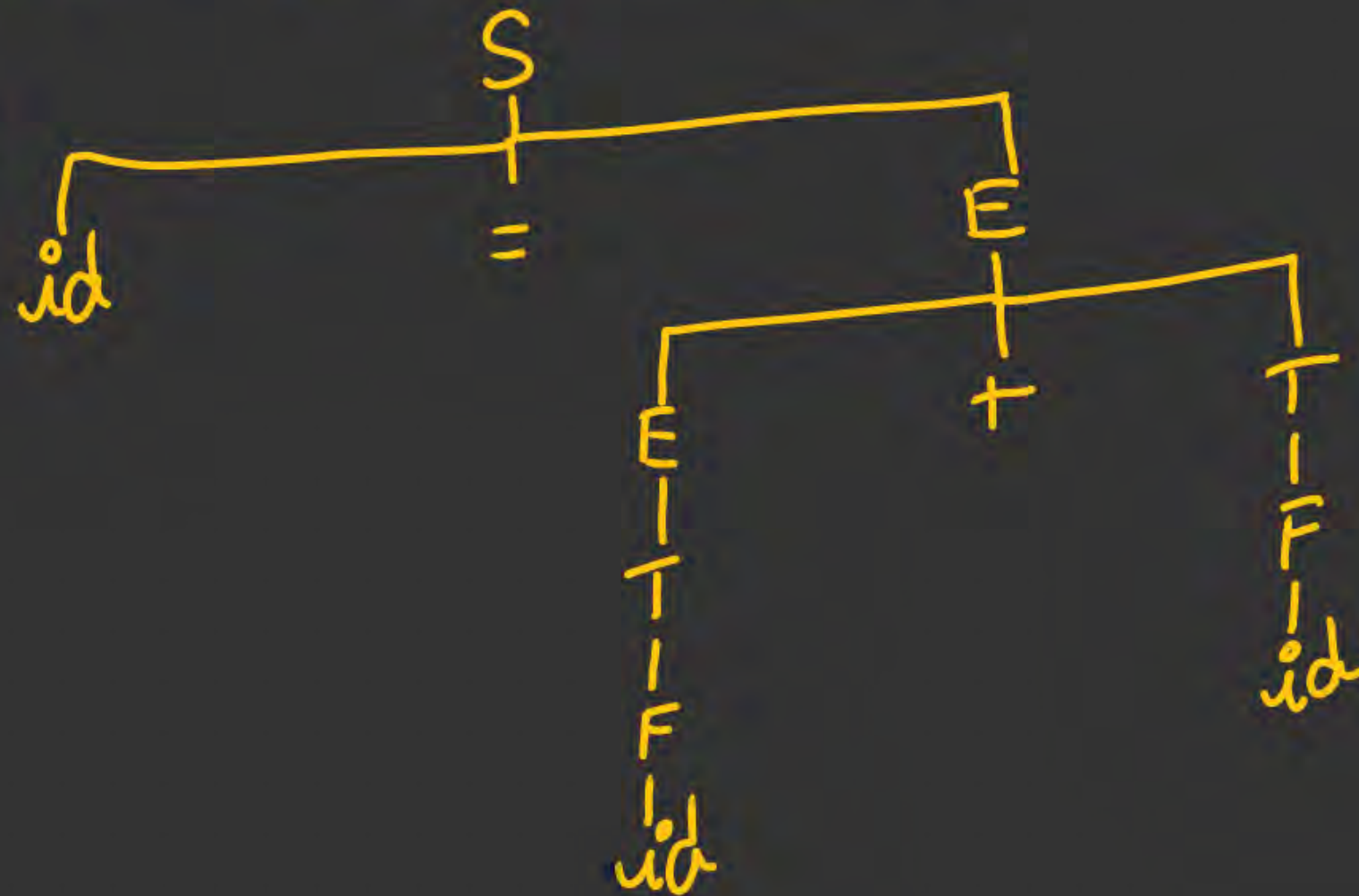
$E \rightarrow E + T \{ E.place = \text{new Temp}(); \text{gen}(E.place = E_1.place + T.place) \}$

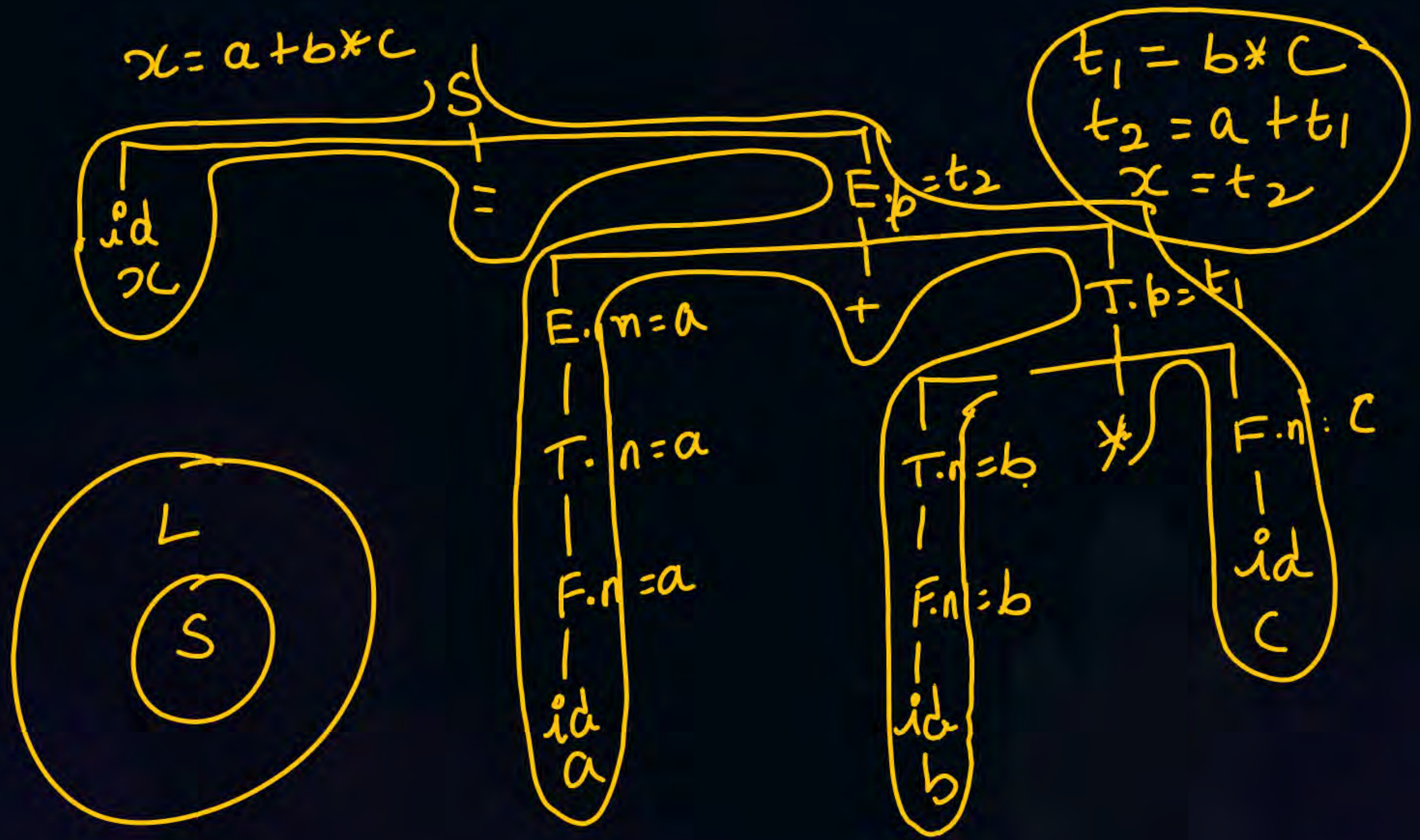
$/ T \{ E.place = T.place \}$

$T \rightarrow T_1 * F \{ \text{gen}(T.place = T_1.place * F.place) \}$

$/ F \{ T.place = F.place \}$

$F \rightarrow id \{ F.place = id.name \}$





$S \rightarrow aaS \quad \{ \text{print 'a'} \}$
 $\quad \quad \quad /bA \quad \{ \text{print 'b'} \}$
 $\quad \quad \quad /b \quad \{ \text{print 'c'} \}$
 $A \rightarrow bCA \quad \{ \text{print 'd'} \}$
 $\quad \quad \quad /cdS \quad \{ \text{print 'e'} \}$

aa b b c c d b

(ccdba) $\rightarrow$ ans



THANK - YOU